Full Stack Development with MERN: Project Documentation

1. Introduction

- **Project Title:** ShopEZ — Resilient E-Commerce Architecture with Context-Adaptive UI Layouts.
- **Team Members & Roles: Shashank(Individual)** (Lead Systems Analyst, Database Architect, and Full-Stack Software Engineer (Responsible for custom Vanilla CSS layout structures, route-guard middleware engineering, and the makeModelProxy database resiliency layer).

2. Project Overview

Purpose

ShopEZ is designed to solve critical operational inefficiencies faced by small-to-medium retail vendors and boutique shop owners when establishing a digital presence. Traditional e-commerce configurations introduce high technical debt, separate management dashboards, and high client-side latency on mobile screens. ShopEZ addresses these issues by delivering a lightweight, single-page application (SPA) that unifies consumer storefronts with administrative metrics into one cohesive, performance-optimized workspace.

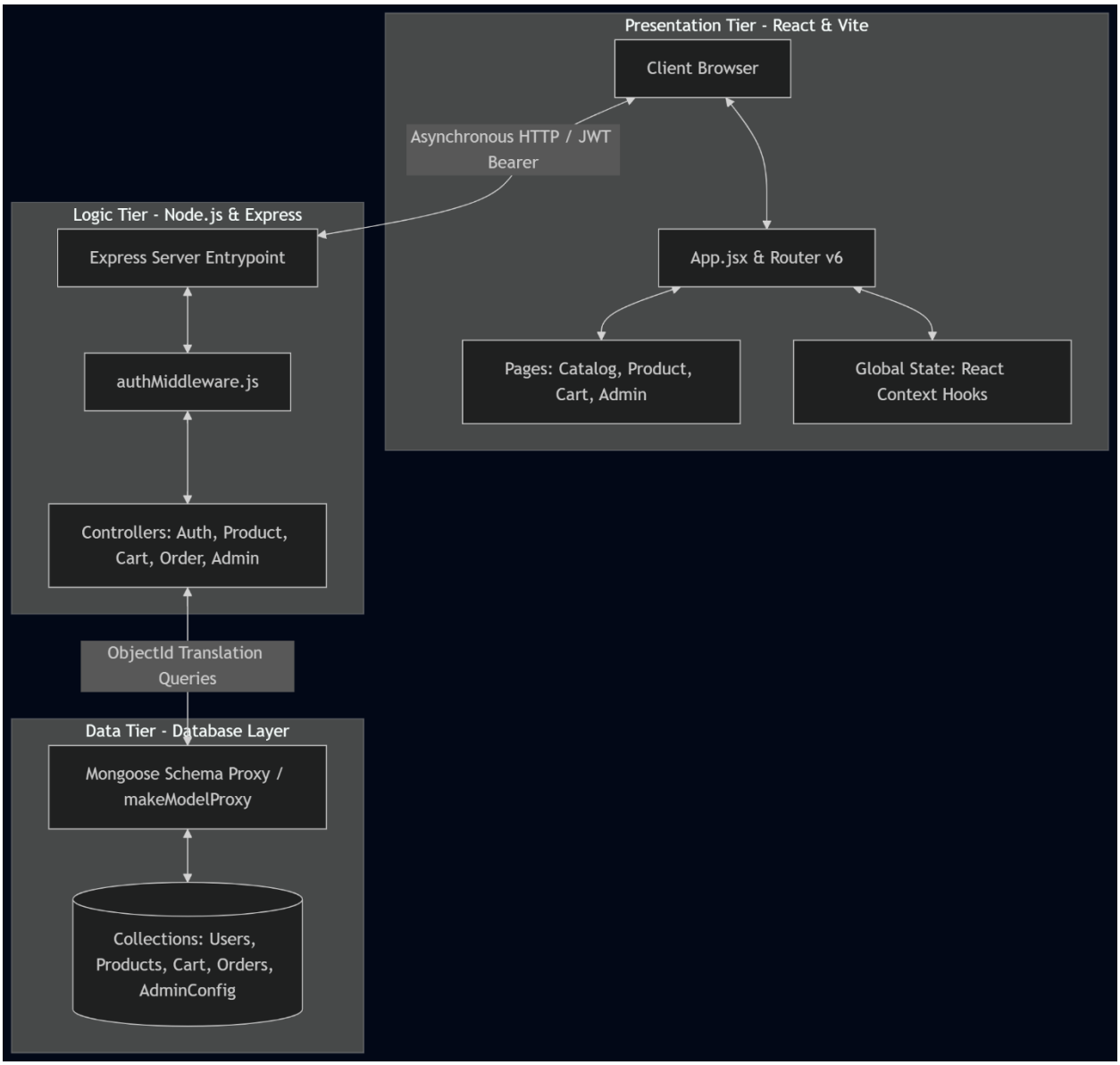
Features

- **Seamless Authentication Infrastructure:** Secures both standard user browsing sessions and administrative privileges using stateless JSON Web Tokens (JWT) verified server-side.
- **Dynamic Content Mapping Engine:** Enables administrators to modify front-page promotional hero banners and active layout categories instantly from the dashboard without modifying source code.
- **Context-Aware UI Layouts:** Automatically evaluates database product categories on the fly to reveal or strip interface items (such as hiding clothing size selectors when rendering electronic products) to lower cognitive load.
- **Absolute Database Resilience:** Incorporates a unique model proxy handler that dynamically shifts database routing to local filesystem JSON files if cloud network timeouts occur.
- **Transparent Transactional Invoicing:** Provides immediate, mathematically clear transaction bills outlining Market Retail Price (MRP), active markdown percentages, and Cash-on-Delivery (COD) status tracking.

3. Architecture

Frontend

The presentation tier is engineered using React.js compiled via Vite to optimize asset compression and minimize client-side paint times. Client-side routing is handled via React Router v6, while persistent user baskets and login profiles are managed globally using the React Context API. Instead of bundling large external utility frameworks, the application relies entirely on highly optimized, native Vanilla CSS variables to drive spacing, glassmorphic layouts (backdrop-filter), and instant dark theme rendering.

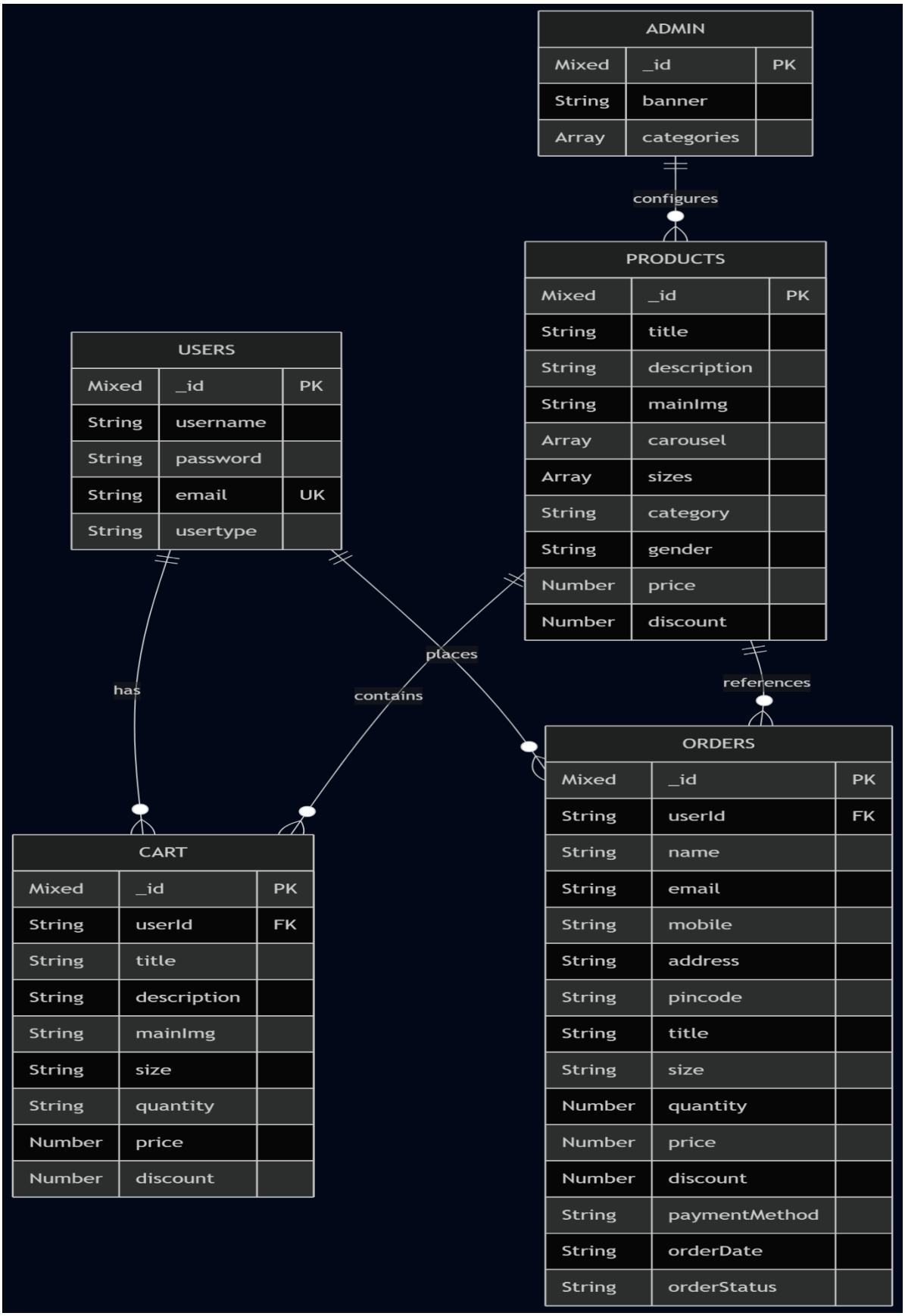


Backend

The logic tier is structured as a stateless Node.js and Express.js RESTful API. It manages application flow through isolated controller modules that handle authentication pipelines, catalog item searching, cart mutations, and checkout sequences. Security constraints are enforced at the routing boundary using specialized middleware handlers to authenticate stateless request keys.

Database

The data tier leverages MongoDB Atlas as its primary cloud data environment, interacting through the Mongoose Object Data Modeling (ODM) framework. The collections are structured into five operational tables (users, admin, products, cart, and orders). To bypass restrictive network firewalls during deployment, the architecture features a custom makeModelProxy constructor layer. If a cloud timeout triggers, this wrapper automatically casts parameter values into ObjectIds and flushes write/read states directly to local seeded JSON backup files on the host file system.


ADMIN
Mixed _id PK
String banner
Array categories
configures
PRODUCTS
Mixed _id PK
String title
String description
String mainImg
Array carousel
Array sizes
String category
String gender
Number price
Number discount
USERS
Mixed _id PK
String username
String password
String email UK
String usertype
has
contains
places
references
CART
Mixed _id PK
String userId FK
String title
String description
String mainImg
String size
String quantity
Number price
Number discount
ORDERS
Mixed _id PK
String userId FK
String name
String email
String mobile
String address
String pincode
String title
String size
Number quantity
Number price
Number discount
String paymentMethod
String orderDate
String orderStatus

4. Setup Instructions

Prerequisites

- Node.js runtime environment (v18.x or higher)
- npm package manager (v9.x or higher)
- MongoDB Atlas Cloud cluster or a local instance of MongoDB Community Server

Installation

1. **Clone the Repository:**

   git clone https://github.com/Shashank-0075/VIP-C2-ShopEZ.git

   cd ShopEZ

2. **Configure Frontend Dependencies:**

```
Bash

cd client
npm install
```

3. **Configure Backend Dependencies:**

```
Bash

cd ../server
npm install
```

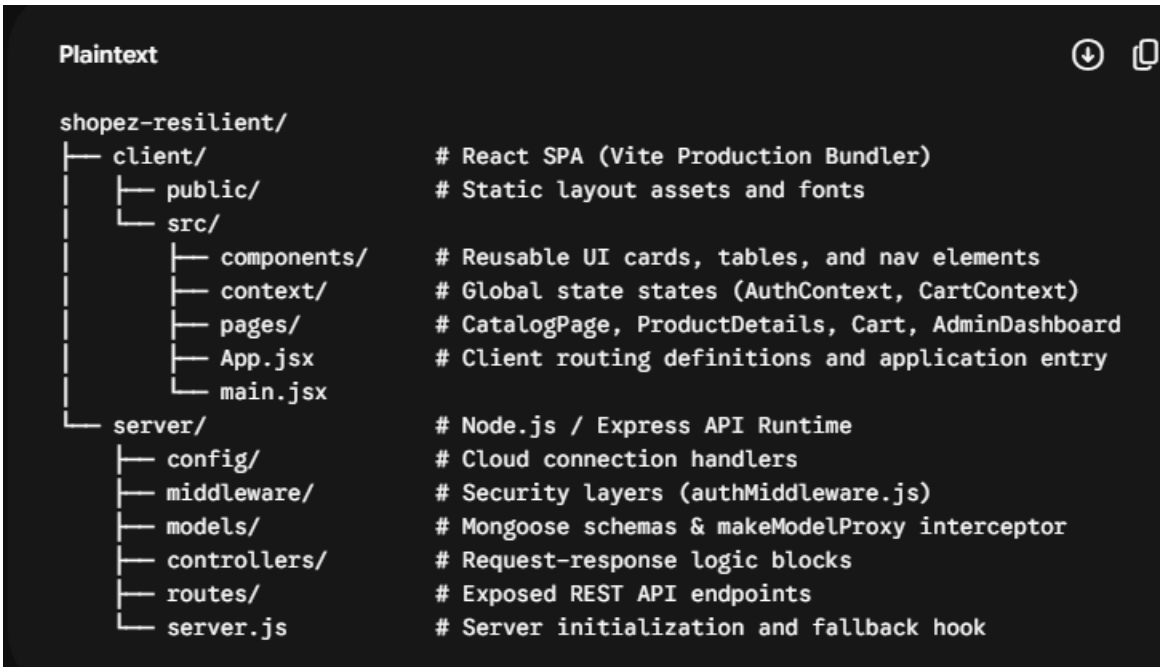
4. **Establish Environment Configurations:** Create a .env file within the /server root workspace directory:

```
Code snippet

PORT=8000
MONGO_URI=mongodb+srv://<admin>:<password>@shopez-cluster.mongodb.net/mainDB
JWT_SECRET=shashank_cryptographic_secret_token_key_2026
```

(Note: If local networks block the database host, the built-in fallback resilience mechanism automatically initializes file-based JSON storage).

5. Folder Structure

The platform organizes source files into a clean repository split to decouple client presentation from server routing mechanics:



```
Plaintext

shopez-resilient/
├── client/                 # React SPA (Vite Production Bundler)
│   ├── public/             # Static layout assets and fonts
│   └── src/
│       ├── components/     # Reusable UI cards, tables, and nav elements
│       ├── context/        # Global state states (AuthContext, CartContext)
│       ├── pages/          # CatalogPage, ProductDetails, Cart, AdminDashboard
│       ├── App.jsx         # Client routing definitions and application entry
│       └── main.jsx
└── server/                 # Node.js / Express API Runtime
    ├── config/             # Cloud connection handlers
    ├── middleware/         # Security layers (authMiddleware.js)
    ├── models/             # Mongoose schemas & makeModelProxy interceptor
    ├── controllers/        # Request-response logic blocks
    ├── routes/             # Exposed REST API endpoints
    └── server.js           # Server initialization and fallback hook
```

6. Running the Application

To run the full application locally during development workflows, execute the following commands in separate terminal interfaces:

- **To Fire Up the Backend REST API (Port 8000):**

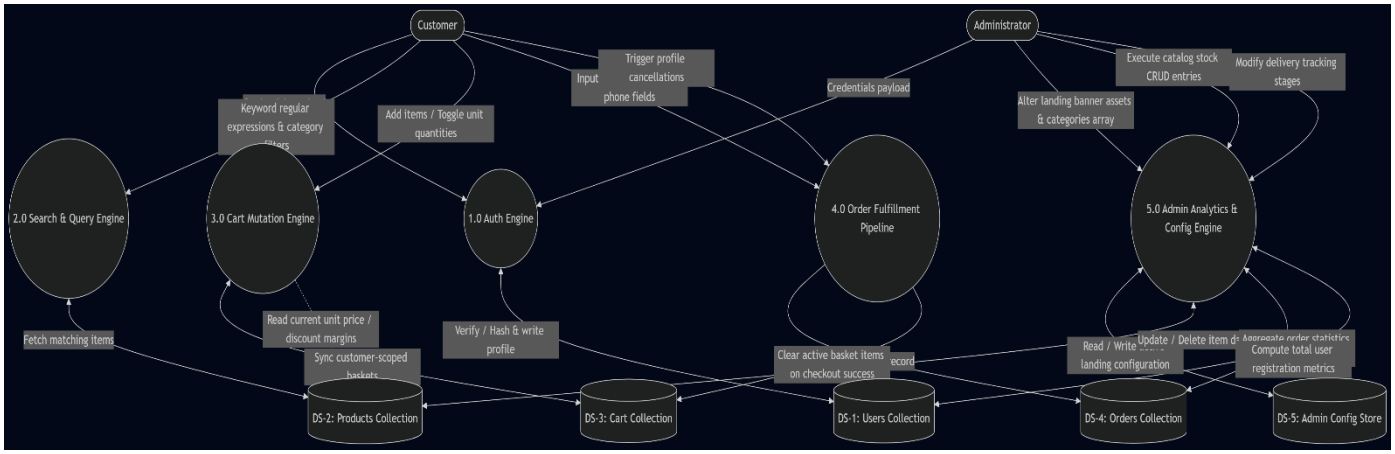
Bash

```
cd server

npm start
```

- **To Fire Up the Frontend Client Client (Port 5173):**

Bash

```
cd client

npm run dev
```



7. API Documentation

All exposed backend routes accept and respond with standardized JSON payloads:

1. Account Session Authentication

- **Method & Endpoint:** POST /api/auth/login
- **Request Payload (JSON):**

JSON

```
{
  "email": "customer@shopez.com",
  "password": "customer123"
}
```

- **Successful Response (200 OK):**

JSON

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "usertype": "Customer",
  "username": "BoutiqueBuyer"
}
```

2. Search Catalog Query

- **Method & Endpoint:** GET /api/products
- **Query Parameters:** ?search=fashion&gender=Men
- **Successful Response (200 OK):**

JSON


```
[
 {
  "_id": "65c2b3e4f5a6b7c8d9e0f1a2",
  "title": "Vintage Denim Jacket",
  "category": "Fashion",
  "price": 1000,
  "discount": 10,
  "sizes": ["M", "L"]
 }
]
```


3. Log Finalized Checkout Transaction

- **Method & Endpoint:** POST /api/orders
- **Headers:** Authorization: Bearer <JWT_TOKEN>
- **Request Payload (JSON):**

JSON


```
{
 "name": "Shashank",
 "email": "shashank@boutique.com",
 "mobile": "9999999999",
 "address": "Hyderabad, Telangana",
 "pincode": "500001",
```

```
 "title": "Vintage Denim Jacket",
 "size": "M",
 "quantity": 1,
 "price": 1000,
 "discount": 10,
 "paymentMethod": "COD"
}
```


- **Successful Response (201 Created):**


```
JSON
{
 "message": "Order created successfully",
 "orderId": "65d3c4e5f6a7b8c9d0e1f2b3"
}
```


8. Authentication

Session control and route protection are implemented using stateless architectures:

- **Password Encryption:** Plaintext user passwords are salted and cryptographically hashed through a 10-round bcryptjs cycle before write operations.
- **Token Distribution:** Successful login execution generates a stateless JSON Web Token signed on the server containing the unique account profile _id and role permissions (usertype).
- **Client Handshaking & Middleware Guarding:** The client caches this token payload in local storage and includes it inside outgoing secure Axios network requests via the HTTP Authorization: Bearer <token> header. Express endpoints pass incoming traffic through authMiddleware.js to cross-examine token signatures, stripping invalid requests and returning a strict 403 Forbidden response to block access.

9. User Interface

The customer storefront interface focuses on high responsiveness and minimal visual clutter:

- **Dynamic Sizing Matrix Toggles:** When rendering items under clothing categories, sizing options show up on the screen. When a user switches to electronics, the UI logic instantly hides these selectors, keeping forms clean.
- **Dynamic Theme Class Injection:** When an administrator authenticates and enters /admin, the application injects an .admin-theme class directly into the document <body> tag. This dynamically overrides active global CSS variable tokens, adapting the interface into a high-contrast dark cockpit dashboard without needing redundant styling templates.

10. Testing

The platform testing strategy balances interface verification with backend constraint auditing:

- **User Acceptance Testing (UAT):** Executed formal test routines (TC-001 through TC-003) validating form verification constraints, theme swaps, and cross-user dataset modification blocks.
- **Automated Postman Routing Diagnostics:** Audited text-matching regex queries, payload validation constraints, and error statuses across all Express controller frameworks.

11. Screenshots or Demo

- **Local Presentation Endpoint:** http://localhost:5173
- **Local API Server Root:** http://localhost:8000

Staging Demo Account Profiles:

- **Standard Shopper Profile:** customer@shopez.com (Password: customer123)

- **System Administrative Gateway:** admin@shopez.com (Password: admin123)

12. Known Issues

- **Cart Parameter Schema Mismatch (BG-001):** The database layout lists quantity counts as a String field, causing legacy client UI code to trigger string concatenation (rendering "12" total items instead of adding them mathematically to show 3). *Status: Resolved by applying hard backend parseInt() sanitizers*.
- **Cross-Tab Synchronicity Lag (BG-002):** Opening the application across concurrent browsing tabs creates a temporary lag where item counts fail to sync in real-time unless the user manually refreshes the second tab window. *Status: Open*.

13. Future Enhancements

- **Cross-Tab Window Synchronization Hooks:** Integrating native browser window storage event listeners to instantly sync cart state updates across active browser tabs without layout lag.
- **Integrated Card Gateway Modules:** Incorporating direct online merchant payment gateway APIs (like Stripe or PayPal) to support a 1.5% to 2.5% transaction commission revenue channel.
- **Localized Redis Caching Layer:** Deploying an in-memory Redis database layer to store high-traffic global data schemas (such as active home banners and categories arrays), reducing the load on MongoDB cloud clusters.